

TUGAS : PENGEMBANGAN DAN ANALISIS SISTEM DISTRIBUSI PESAN & FILE BERBASIS HYBRID SOCKET

Mata Kuliah: Jaringan Komputer Lanjut (S2 Ilmu Komputer)

Materi Referensi: Kurose & Ross, *Computer Networking: A Top-Down Approach (9th Edition)*, Section 2.6 & Chapter 2 Review Problems.

1. Deskripsi Tugas & Spesifikasi Sistem

Mahasiswa diminta untuk membangun aplikasi *Client-Server* berbasis *Command Line Interface* (CLI) atau *Web API* sederhana menggunakan bahasa pemrograman **Python 3** atau **C/C++**. Aplikasi ini menggabungkan layanan transmisi teks berbasis TCP (konkuren) dan layanan *pinging/heartbeat* status server berbasis UDP.

Spesifikasi Fitur Utama:

1. **Multi-Threaded / Asynchronous TCP Server (TCPServer.py):**
 - Berjalan pada port tertentu (misal: 12000).
 - Dapat melayani N koneksi *client* secara bersamaan (menggunakan *threading*, *multiprocessing*, atau *asyncio*).
 - **Manajemen Sesi & Frame:** Mengimplementasikan protokol layer aplikasi buatan sendiri untuk menghindari masalah *Byte Stream Boundary / HOL Blocking*. Perlu dipastikan *client* dan *server* dapat memisahkan pesan dengan jelas (misalnya menggunakan panjang *header fixed-length* atau *delimiter*).
 2. **UDP Health Checker / Heartbeat Server (UDPServer.py):**
 - Berjalan di samping server TCP untuk menerima pesan *ping* berkala dari *client* guna mengukur RTT (Round Trip Time) dan *packet loss*.
 - Mengimplementasikan batas *timeout* 1 detik di sisi *client* untuk menangani *unreliable packet delivery*.
-

2. Lembar Soal & Pertanyaan Analisis (Bobot 100%)

Bagian A: Implementasi Pemrograman Soket (Bobot 40%)

1. **Layanan TCP multi-client (20%):** Tuliskan potongan kode *welcoming socket* dan cara *server* Anda menangani `accept()` untuk spawn thread baru bagi tiap *client* yang terhubung. Jelaskan perbedaan *lifecycle* antara *welcoming socket* (`serverSocket`) dan *connection socket* (`connectionSocket`).
2. **Layanan UDP Pinger (20%):** Tuliskan potongan kode *client* UDP yang mengosongkan antrean atau menangani paket yang *lost* menggunakan

`settimeout(1.0)` dan menghitung perkataan RTT dari 10 kali pengiriman pesan.

Bagian B: Analisis Mendalam & Arsitektur Jaringan (Bobot 60%)

Jawablah pertanyaan-pertanyaan analitis berikut berdasarkan konsep bab 2 buku Kurose:

1. Byte-Stream vs Message Boundary (Bobot 15%):

- Berdasarkan pemahaman Anda mengenai API soket TCP (`SOCK_STREAM`) dan UDP (`SOCK_DGRAM`), jelaskan mengapa TCP dianggap sebagai *byte-stream channel* tanpa batasan pesan (*no message boundaries*) sedangkan UDP mempertahankan *message boundaries*.
- Apa dampak teknis hal ini saat *client* TCP mengirimkan 3 buah string pesan berturut-turut melalui soket `send()` tanpa *delay*? Bagaimana rancangan protokol aplikasi Anda untuk menyelesaikan kendala ini?

2. Analisis Skalabilitas Socket (Bobot 15%):

- Jika *server* TCP Anda melayani N *client* secara bersamaan dari IP yang berbeda-beda, berapa jumlah total soket (*file descriptor*) yang dibuka oleh proses *server* tersebut?
- Mengapa *server* UDP hanya membutuhkan 1 soket tunggal meskipun melayani permintaan dari N *client* yang berbeda? Jelaskan implikasinya terhadap konsumsi memori OS dan *port binding*.

3. Evolusi Transport & QUIC (Bobot 15%):

- Buku Kurose menyebutkan pengenalan HTTP/3 yang berjalan di atas QUIC. Jelaskan mengapa QUIC memilih berjalan di atas *socket* UDP (`SOCK_DGRAM`) dan bukan membuat *transport protocol* baru dari awal atau menumpang di TCP.
- Bagaimana QUIC menyelesaikan masalah *Head-of-Line (HOL) Blocking* yang terjadi pada TCP tingkat transport?

4. Skenario Bind Explicit pada Client (Bobot 15%):

- Secara umum, *client* membiarkan sistem operasi memilih *port* lokal secara acak (*ephemeral port*). Apa yang terjadi jika pada kode `UDPClient.py` Anda secara eksplisit menambahkan perintah `clientSocket.bind('', 5432)` sebelum mengirim pesan?
- Apakah *server* masih bisa membalas pesan tersebut? Apa potensi kendala (*conflict*) yang terjadi di sisi *client* jika terdapat 2 instansiasi aplikasi *client* yang berjalan bersamaan di host yang sama?

3. Format Pengumpulan Tugas

1. **Source Code:** Berupa repositori GitHub berisi file `.py` / `.cpp` lengkap dengan instruksi cara menjalankan (*README.md*).
2. **Laporan Analisis (PDF):** Maksimal 5 halaman yang memuat tanggapan pertanyaan Bagian B serta bukti tangkapan layar paket yang ditangkap

- menggunakan **Wireshark** saat interaksi TCP & UDP terjadi.
3. *Link* github ditulis di elok sebagai jawaban tugas